



Tensor Gaussian Processes: Efficient Solvers for Nonlinear PDEs

Qiwei Yuan¹, Zhitong Xu¹, Yinghao Chen², Yiming Xu², Houman Owhadi³, Shandian Zhe¹

²Department of Mathematics, University of Kentucky; ³Department of Computing and Mathematical Sciences, California Institute of Technology



Theory:

Convergence:

Lemma 4.4. Let $u^* \in \mathcal{U}$ denote the unique strong solution of (1), and suppose Assumption 4.3 holds. Let $\mathcal{M} = \mathcal{M}_\Omega \cup \mathcal{M}_{\partial\Omega}$ be a set of collocation points, with $\mathcal{M}_\Omega \subset \Omega$ and $\mathcal{M}_{\partial\Omega} \subset \partial\Omega$. Assume the Voronoi diagram induced by $\mathcal{M}$ has a uniformly bounded aspect ratio across all cells. Define the fill-distances $h_\Omega := \sup_{\mathbf{x} \in \Omega} \inf_{\mathbf{x}' \in \mathcal{M}_\Omega} |\mathbf{x} - \mathbf{x}'|$ and $h_{\partial\Omega} := \sup_{\mathbf{x} \in \partial\Omega} \inf_{\mathbf{x}' \in \mathcal{M}_{\partial\Omega}} \rho_{\partial\Omega}(\mathbf{x}, \mathbf{x}')$, where $|\cdot|$ is the Euclidean distance, and $\rho_{\partial\Omega}$ is a geodesic distance defined on $\partial\Omega$. Set $h = \max(h_\Omega, h_{\partial\Omega})$. Suppose each RKHS $\mathcal{G}^i$, to which the factor functions in dimension i belong, is associated with a universal kernel, and that $\mathcal{U} = \mathcal{G}^1 \otimes \dots \otimes \mathcal{G}^d$. Then, for any arbitrarily small $\varepsilon > 0$, with a sufficiently large R and an appropriate δ , the optimization (learning) problem (7) under the CP format (4), always has a minimizer u^1 , and this minimizer satisfies $\|u^1 - u^*\|_{H^1(\Omega)} \lesssim \varepsilon$ as $h \rightarrow 0$.

Motivation & Background :

• **The Problem:** Solving Partial Differential Equations (PDEs) is central to science/engineering

• **Previous Work:**

- **Traditional numerical methods(FD, FEM, ...):**
 - Professional skills in mesh design required;
 - Expensive runtime cost due to iterative computing;
 - Challenge on high-dimensional Problems;
- **Machine Learning(ML) Solvers-ease of implementation and competitive accuracy:**
- **Physics-Informed Neural Networks (PINNs)-approximate the PDE solution using a (deep) neural network**
Due to stochastic optimization, training (solving) process is often lengthy and inefficient; hyper parameter tuning often needs long-time tedious trial and error.

Gaussian Process and Kernel-Based Solvers:

- Mathematical Principled & uncertain quantification
- **DAKS(Derivative-Augmented Kernel Solver) Chen et al. (2021 C)-place a GP prior over the solution and all its derivative**
Runtime and memory usage may explode as M(number of collocation points) increase due to large covariance matrix.
- **SKS(Simple Kernel-based Solver) Xu et al., (2025)**

approximate the solution using $u(\mathbf{x}; \mathbf{u}_M) = \kappa(\mathbf{x}, \mathcal{M}) \mathbf{K}_{MM}^{-1} \mathbf{u}_M$;

adopt a product kernel of the form $\kappa(\mathbf{x}, \mathbf{x}') = \prod_j \kappa_j(x_j, x'_j)$, the covariance matrix admits a Kronecker product structure: $\kappa(\mathcal{M}, \mathcal{M}) = \mathbf{K}_1 \otimes \dots \otimes \mathbf{K}_d$

Rely on stochastic optimization; estimation on grid grows exponentially as dimension increasing

Our Proposal: TGPS, a tensor-GP based PDE solvers

Problem Setting:

for each input dimension i , introduce 1D factor:

$$f_r^i : \Omega_0 \subset \mathbb{R} \rightarrow \mathbb{R} \in \mathcal{G}^i \ (1 \leq r \leq R_i), \quad (3)$$

where $1 \leq i \leq d$, and $\mathcal{G}^i$ is RKHS induced by $\kappa_i(\cdot, \cdot)$

With a set of inducing locations $\gamma_i = (\gamma_1, \dots, \gamma_{N_i})^\top$ in each dimension i , we get:

$$f_r^i(x_i) = \kappa_i(x_i, \gamma_i) \mathbf{K}_i^{-1} \boldsymbol{\eta}_r^i, \quad (4)$$

The loss is designed as:

$$\begin{aligned} \text{minimize } \mathcal{F}(u(\mathbf{x}; \{\boldsymbol{\eta}_r^i\}); \alpha_1, \alpha_2) &:= \sum_{i=1}^d \sum_{r=1}^R \|f_r^i\|_{\mathcal{G}^i}^2 \\ &+ \alpha_1 \left[\frac{1}{M_\Omega} \sum_{m=1}^{M_\Omega} (\mathcal{P}(u)(\mathbf{x}_m) - a(\mathbf{x}_m))^2 - \delta^2/2 \right] \\ &+ \alpha_2 \left[\frac{1}{M - M_\Omega} \sum_{m=M_\Omega+1}^M (\mathcal{B}(u)(\mathbf{x}_m) - b(\mathbf{x}_m))^2 - \delta^2/2 \right] \end{aligned} \quad (5)$$

where $\alpha_1, \alpha_2 > 0$ represent regularization strength, and δ can be simply set to zero.

Methodology :

• **Model Architecture:**

- Approximates the solution $u(\mathbf{x})$ via a Tensor Decomposition with 1D factor functions.
- Each factor function along dimension d is modeled by a 1D Gaussian Process.

CANDECOMP/PARAFAC(CP) Decomposition:

$$u(x_1, \dots, x_d) = \sum_{r=1}^R \prod_{i=1}^d f_r^i(x_i) = (\mathbf{f}^1(x_1) \circ \dots \circ \mathbf{f}^d(x_d))^\top \mathbf{1}, \quad (4)$$

where each $\mathbf{f}^i(x_i) = (f_1^i(x_i), \dots, f_R^i(x_i))^\top$, and $\circ$ is Hadamard (element-wise) product.

Tensor-Ring(TR) Decomposition:

$$u(x_1, \dots, x_d) = \text{Trace}(\mathbf{F}^1(x_1) \mathbf{F}^2(x_2) \dots \mathbf{F}^d(x_d)). \quad (5)$$

Expressiveness:

Lemma 4.1. Suppose that $u^* \in H^k(\Omega)$ for some $k \in \mathbb{N}$, where $H^k(\Omega)$ denotes the Sobolev space consisting of functions whose weak derivatives up to the order of k have finite L^2 norms. Given an arbitrary error $0 < \varepsilon < 1$, if we model u as the CP format in (4) and let $\bar{R} = (\frac{\sqrt{d}}{\varepsilon})^{\frac{d-1}{k}}$, then

$$\min_{R \leq \bar{R}, u_r^i \in L^2(\Omega_0)} \|u^* - u\|_{L^2(\Omega)} \lesssim \varepsilon. \quad (13)$$

Lemma 4.2. Suppose that $u^* \in H^{k+1}(\Omega)$ for some $k \in \mathbb{N}$, where $H^k(\Omega)$ denotes the Sobolev space consisting of functions whose weak derivatives up to the order of k have finite L^2 norms. Given an arbitrary error $0 < \varepsilon < 1$, if we model u as the TR format in (5) and let $\bar{R} = (\frac{\sqrt{d}}{\varepsilon})^{\frac{d}{k}}$, then

$$\min_{R \leq \bar{R}, u_r^i \in L^2(\Omega_0)} \|u^* - u\|_{L^2(\Omega)} \lesssim \varepsilon. \quad (14)$$

• **Handling Nonlinearity:**

- Partial Freezing: freeze part of the nonlinear terms from the previous iteration, leaving only a linear component;

$$u(u^2 - 1) \approx u \cdot ((u^{\text{prev}})^2 - 1)$$

- Newton's method(Tangent Linearization): replace the nonlinear terms with their first-order Taylor expansions.

$$u^3 \approx (u^{\text{prev}})^3 + 3(u^{\text{prev}})^2(u - u^{\text{prev}})$$

• **Alternating Least Squares(ALS):**

- The solution approximation is multilinear in the inducing values;
- Derivatives of the solution preserve the same multilinear structure;
- With linearization method, any PDE can be transformed as linear.

• **Advantage:**

- **Memory Efficiency:**scale linearly with both the PDE dimension and the number of collocation points-not only in covariance matrix computation and storage but also in the number of trainable parameters;
- **Runtime Efficiency:** compared to traditional stochastic optimization, ALS yields far greater efficiency and achieving accurate approximations with only a small number of iteration.

Detail

$$u_{x_1 x_1} + u_{x_2 x_2} + u(u^2 - 1) = a(x_1, x_2), \quad (9)$$

where $a(x_1, x_2)$ is the source function.

With $\mathbf{H}_i = [\boldsymbol{\eta}_1^i, \dots, \boldsymbol{\eta}_{N_i}^i] \in \mathbb{R}^{N_i \times R}$, set $\mathbf{f}^i(x_i) = (\kappa_i(x_i, \gamma_i) \mathbf{K}_i^{-1} \mathbf{H}_i)^\top = \mathbf{H}_i^\top \mathbf{w}_i(x_i)$ where $\mathbf{w}_i(x_i) = \mathbf{K}_i^{-1} \kappa_i(\gamma_i, x_i)$, and

$$u(x_1, \dots, x_d) = \langle \odot_{i=1}^d \mathbf{H}_i^\top \mathbf{w}_i(x_i), \mathbf{1} \rangle, \quad (10)$$

Then $u(x_1, x_2) = \mathbf{w}_1(x_1)^\top \mathbf{H}_1 \mathbf{H}_2^\top \mathbf{w}_2(x_2)$

The partial derivative over the solution approximation is: $\partial_{x_{j_1} \dots x_{j_K}} u = \langle \odot_{i=1}^d \mathbf{H}_i^\top \hat{\mathbf{w}}_i(x_i), \mathbf{1} \rangle$ where $\hat{\mathbf{w}}_i(x_i)$ is the derivative of $\mathbf{w}_i(x_i)$ if $x_i \in \{x_{j_1}, \dots, x_{j_K}\}$ and otherwise $\hat{\mathbf{w}}_i(x_i) = \mathbf{w}_i(x_i)$

We get:

$$\begin{aligned} u_{x_1 x_1} &= (\mathbf{w}_1''(x_1))^\top \mathbf{H}_1 \mathbf{H}_2^\top \mathbf{w}_2(x_2), \\ u_{x_2 x_2} &= \mathbf{w}_1(x_1)^\top \mathbf{H}_1 \mathbf{H}_2^\top \mathbf{w}_2''(x_2), \end{aligned} \quad (11)$$

where $\mathbf{w}_1''(x_1) = \partial^2 \mathbf{w}_1 / \partial x_1^2$ and $\mathbf{w}_2''(x_2) = \partial^2 \mathbf{w}_2 / \partial x_2^2$.

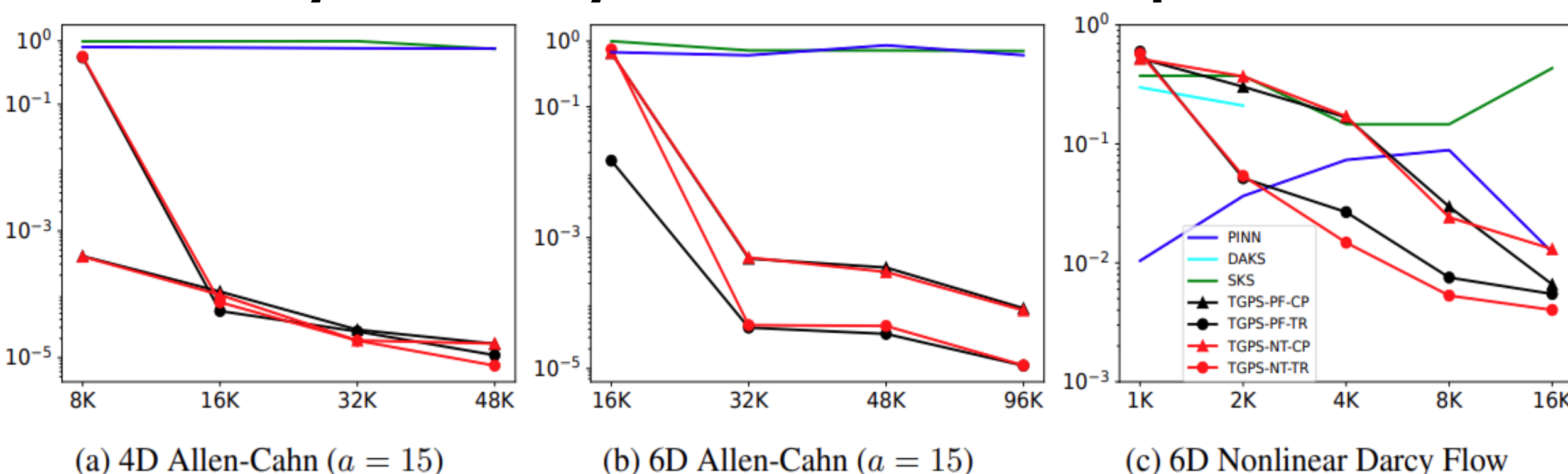
The PDE can be transformed to:

$$(\mathcal{P}(u)(\mathbf{x}_m) - a(x_m))^2 = (\text{tr}(\mathbf{B}_m^\top \mathbf{H}_1) - a(\mathbf{x}_m))^2 \quad (12)$$

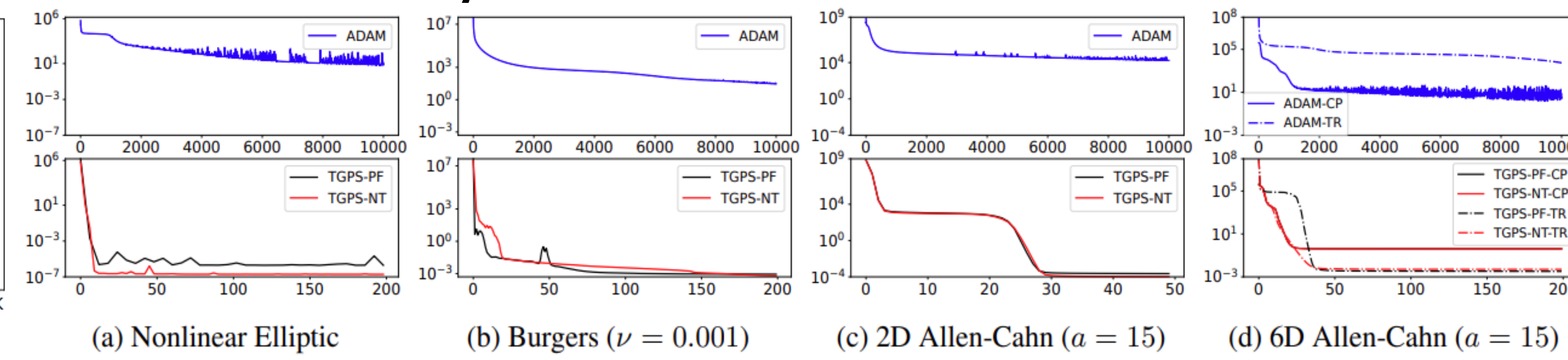
where $\mathbf{B}_m = (\mathbf{w}_1''(x_{m1}) \mathbf{w}_2''(x_{m2}) + \mathbf{w}_1(x_{m1}) \mathbf{w}_2''(x_{m2}))^\top \cdot \mathbf{H}_2$,

Experimental Results:

• **Memory Efficiency: 4D & 6D PDE Comparison**



• **Runtime Efficiency-ALS vs. Stochastic**



• **Runtime Efficiency-Runtime Comparison**

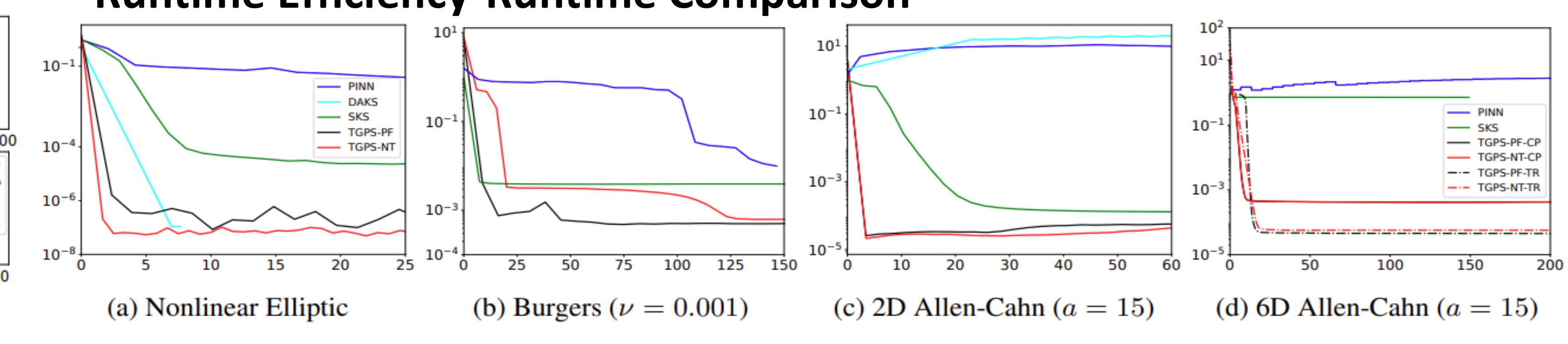


Figure 1: Relative L^2 error vs. the number of collocation points in solving higher-dimensional PDEs.

Figure 2: Training curves: Training loss vs. Number of iterations.

Figure 3: Running time (in seconds) vs. Relative L^2 error.